1. (12 points) Suppose that `s = "banana"`. What are the values of the following Python expressions?

   (a) `set(s)`

   (b) `{c: s.count(c) for c in s}`

   (c) `sorted(s) # Hint: sorted() always returns a list`

   (d) `s[1:4]`

2. (8 points) Consider the following list:

```
students = [
    {"name: "Alice Zimmer", "gpa": 3.5, "credits": 36},
    {"name: "Brian Young", "gpa": 3.8, "credits": 111},
    {"name: "Carolyn Xiang", "gpa": 2.7, "credits": 55},
    {"name: "Declan West", "gpa": 1.9, "credits": 24}
]
```

   (a) What Python expression would return the GPA of the last student in the list?

   (b) Suppose that at the end of the semester, Alice Zimmer completes 15 more credits and her GPA increases to 3.6. Write one or two lines of Python code to update the information in `students`.

3. (12 points) Looking at the same list of students as the previous question, what are the values of the following expressions?

   (a) `"Alice Zimmer" in students`

   (b) `"credits" in students[0]`

   (c) `len(students)`

   (d) `len(students[0])`

4. (10 points) Consider the following function.

```
------------------------------------------------------------
def mystery(number_list):
    output = []
    for i in range(1, len(number_list)):
        current, previous = number_list[i], number_list[i-1]
        output.append(current - previous)
    return output
------------------------------------------------------------
```

(a) What does `mystery([1, 5, 0, 8])` return?

(b) Which of the following best describes this function?

A. It is a pure function because it has no side effects.

B. It is not a pure function because the input is a mutable list.

C. It is a pure function because it returns a list.

D. It is not a pure function because it returns a list.

5. (6 points) Consider the following code and answer the questions below.

```
------------------------------------------------------------
meal_requests = {"breakfast": 0, "lunch": 0, "dinner": 0}

def process_requests(guest_order):
    output = []
    for meal in ["breakfast", "lunch", "dinner"]:
        if meal in guest_order:
            meal_requests[meal] += 1
            output.append(meal)
    return output

s = "I would like breakfast and dinner, but not lunch."
result = process_requests(s)
------------------------------------------------------------
```

(a) What is the value of `result` after this code runs?

(b) What is the value of `meal_requests` after this code runs?

6. (10 points) Consider the following program.

```
-------------------------------------------------------------------------------
class Family:
    def __init__(self, members):
        self.members = members
    def add_years(self, years):
        for person in self.members:
            person.age += years

class Person:
    def __init__(self, name, age):
        self.name, self.age = name, age
    def __str__(self):
        return f"{self.name} is {self.age} years old."

my_family = Family([Person("Dad", 42), Person("Mom", 39), Person("Junior", 12)])
-------------------------------------------------------------------------------
```

(a) How would you call the `add_years` method to make everyone in `my_family` 3 years older?

(b) Define a method called `print_family` for the `Family` class that would print out the information about all of the members of a family. For example, when you apply the `print_family` method to `my_family`, you should get the following result:

```
Dad is 42 years old.
Mom is 39 years old.
Junior is 12 years old.
```

7. (10 points) Write a class named `Book` that represents books in a library. Each book has the following attributes:

- `title` - the title of the book.
- `author` - the author of the book.

The `Book` class should have an initializer that accepts the book's title and author. The class should also have a `__str__` method that returns a string describing both the title and author of the book (e.g. "Adventures of Huckleberry Finn by Mark Twain").

8. (8 points) Suppose that the following program is saved in a file called `reverse.py`.

```
-- reverse.py ------------------------
def reverse_string(word):
    new_string = ""
    for char in word:
        new_string = char + new_string
    return new_string

if __name__ == "__main__":
    place = "Hampden-Sydney"
    print(reverse_string(place))
-------------------------------------
```

Another program called `test.py` is in the same folder as `reverse.py` and contains these lines of code.

```
-- test.py --------------------------
import reverse

print(reverse.reverse_string("test"))
-------------------------------------
```

(a) What will happen if you run `reverse.py`?

A. It will print `yendyS-nedpmaH` and `tset`.
B. It will not print anything.
C. It will just print `yendyS-nedpmaH`.
D. It will just print `tset`.

(b) What will happen if you run `test.py`?

A. It will print `yendyS-nedpmaH` and `tset`.
B. It will not print anything.
C. It will just print `yendyS-nedpmaH`.
D. It will just print `tset`.

9. (4 points) Why should we use the keyword `with` when we open a file (like in the following example)?

```
--------------------------------
with open("example.txt") as file:
    file_text = file.read()
--------------------------------
```

A. The `with` keyword lets us open the file.
B. The `with` keyword lets us read the contents of the file as a string.
C. The `with` keyword will close the file when we are done.
D. The `with` keyword lets us both read from and write to the file.

10. (20 points) Consider the following string with words separated by spaces (no capitalization or punctuation).

```
"we dont need no education we dont need no thought control no we dont"
```

A lot of the words in the string appear in more than one location. For example, `"we"` appears at position 0, 5, and 12.

(a) Write a function called `get_positions(word, string)` that returns a list of all the positions where a word appears in a string. You can assume that the string contains only lower case words separated by single spaces and no punctuation. Hint: don't forget that you can use the `.split()` method to separate a string into a list of substrings.

```
def get_positions(word, string):
    # Insert your code below.
```

(b) Write a function called `all_positions(string)` that inputs a string of lower case words separated by spaces, and returns a dictionary with keys corresponding to the words in the string, and values that are lists of the positions where those words appear. For example, if you input the string above, then your function should return:

```
{"we": [0, 5, 12], "dont": [1, 6, 13], "need": [2, 7], "no": [3, 8, 11],
           "education": [4], "thought": [9], "control": [10]}
```

If it helps, your `all_positions` function can call `get_positions`.

```
def all_positions(string):
    # Insert your code below.
```